Model Predictive Control

Course notes, part I

Process Identification and Control, academic year 2026/2027
Sapienza University of Rome, DIAG

Version of 27 October 2026 — to be completed after the lesson on task scheduling

These notes cover the first part of the course, on model predictive control. They follow the route of the lessons, from the idea of the receding horizon to the theory of optimality, feasibility and stability, and then to the case studies, but they are written to be read on their own. Each chapter ends with the references for a deeper study; they are the same as those given on the course website, with the same section and page numbers. The interactive laboratories mentioned in the text are on the website, in the lesson diary.

Contents

1 The receding-horizon idea **3**
1.1 What a controller has to do . . . 3
1.2 The receding horizon . . . 3
1.3 The nominal MPC algorithm . . . 3
1.4 Why MPC, and at what price . . . 4
1.5 Centralized and decentralized control . . . 4
1.6 Hard and soft constraints . . . 4

2 Models and the optimization problem of MPC **5**
2.1 State-space models . . . 5
2.2 The problem solved at every step . . . 5
2.3 The ideal problem: infinite horizon . . . 6

3 Optimality: a finite window for an infinite problem **7**
3.1 The value function and Bellman's principle . . . 7
3.2 Dual-mode control . . . 7
3.3 The terminal cost as the cost-to-go . . . 8
3.4 From one step to the next . . . 8

4 Recursive feasibility **9**
4.1 Feasible sets and invariance . . . 9
4.2 The proof . . . 9
4.3 Choosing the terminal set . . . 10

5 Stability **10**
5.1 Definitions . . . 11
5.2 Linear systems . . . 11
5.3 Lyapunov functions . . . 11
5.4 Stability of MPC . . . 12
5.5 The role of each ingredient . . . 13

6 Economic MPC and the design of the cost **14**
6.1 Tracking and economic objectives 14
6.2 Several goals: the weighted sum 14
6.3 The terminal value of a stored resource 14

7 Writing logic as constraints **15**
7.1 Indicator variables and the big-M constraint 15
7.2 Mutually exclusive events 16
7.3 Semi-continuous variables 16
7.4 One start per job: one-hot variables 16
7.5 The product of a real variable and a binary 16
7.6 Maxima and absolute values 17
7.7 Soft constraints and slack variables 17

8 Case studies **17**
8.1 The charging service area 18
8.2 Building energy management 19
8.3 An aggregator of electric vehicles 20
8.4 Traffic lights at an intersection 21
8.5 Task scheduling in an assembly line 23

References **24**

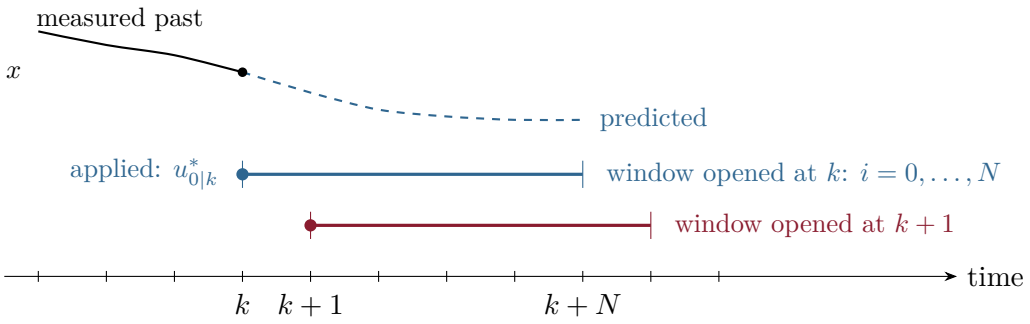


Figure 1: The receding horizon. At time k the controller measures x_k, plans the inputs over the next N steps, applies the first one and discards the rest. At $k+1$ it starts again with a window moved one step ahead.

1 The receding-horizon idea

1.1 What a controller has to do

A *process* is a system whose evolution in time we want to influence: a tank whose level has to be kept constant, a storage system whose charge has to follow the needs of a building, a traffic light whose colours have to keep the queues short. We act on the process through its *control inputs* and we observe it through its *measurements*. A *controller* is the rule that, at every sampling time, turns the measurements into the next value of the inputs.

Classical controllers, such as the PID, compute the input from the current error with a fixed formula. They are simple and robust, but they do not know the limits of the process — a pump that cannot exceed its rated power, a tank that cannot be emptied below a minimum level — and they do not look ahead. Model predictive control (MPC) is a different way of writing a controller: at every sampling time it *plans* the inputs over a window of future steps by solving an optimization problem, and it uses only the first step of the plan.

1.2 The receding horizon

Fix a sampling time T and call $k = 0, 1, 2, \dots$ the sampling instants. At time k the controller measures the state x_k of the process, and considers the next N steps, the *prediction horizon*. Inside the window it uses a second time index, $i = 0, 1, \dots, N$, relative to the beginning of the window: the quantity predicted for step i of the window opened at time k is written with the double index $i|k$, as in $x_{i|k}$ and $u_{i|k}$. The absolute time of that step is $k + i$.

The controller then solves an optimization problem whose unknowns are the inputs over the window, $u_{0|k}, \dots, u_{N-1|k}$, and whose cost measures how well the predicted trajectory meets the control goal. Of the optimal sequence only the first element is applied to the process, $u_k = u^*_{0|k}$. At time $k+1$ the state is measured again and the whole procedure is repeated over a window that has moved one step forward. This is the *receding horizon*: the window recedes as time advances (fig. 1).

1.3 The nominal MPC algorithm

The procedure can be written as an algorithm. The adjective *nominal* means that the controller trusts its model completely: no disturbances, no model errors, a state that can be measured.

Nominal MPC. At every sampling time k:

1. measure the state x_k of the process;

2. solve the optimization problem $\mathcal{P}_N(x_k)$ over the window $i = 0, \dots, N$, obtaining the optimal input sequence $\mathbf{u}_k^* = \{u_{0|k}^*, \dots, u_{N-1|k}^*\}$;
3. apply $u_k = u_{0|k}^*$ to the process and discard the rest of the sequence;
4. wait for the next sampling time and go back to step 1.

The first input of the optimal sequence depends on the measured state: $u_{0|k}^* = \kappa_N(x_k)$. MPC is therefore a *feedback* controller, not an open-loop plan computed once, and κ_N is its control law. It is a law that is never written down explicitly: it is evaluated, at every step, by solving an optimization problem. This has two consequences that run through the whole course. First, MPC is a nonlinear controller even when the process is linear, because κ_N is the solution of a constrained problem. Second, the time needed to solve the problem must be much smaller than the sampling time T, which is the main practical limit of the technique.

1.4 Why MPC, and at what price

The strength of MPC is that the goals of the control problem are written *directly* into the optimization problem: the cost says what we want, the constraints say what is admissible. Limits on inputs and states — the power of a pump, the capacity of a battery, the maximum number of vehicles a queue can hold — are constraints, not afterthoughts; several inputs and several outputs are handled at once; and known information about the future, such as a forecast of the demand or the schedule of a tariff, enters the plan naturally. The formulation is also easy to explain to whoever operates the process, because every term has a meaning.

The price is twofold. The controller needs a *model* of the process, good enough for its predictions to be useful over the window; obtaining such a model is the subject of the third part of the course. And it needs an optimization problem to be solved at every sampling time, which rules out, or makes expensive, the cases in which the sampling time is very short or the problem is very large.

The problems solved by MPC belong to two families. When all the decision variables are continuous and the cost and the constraints are convex — a quadratic cost with linear constraints is the typical case — the problem is a *continuous program*, solved reliably and fast. When some decisions are logical — start a machine or not, turn a light green or not — they are represented by binary variables and the problem becomes a *mixed-integer program*, much harder in general. Both families appear in the case studies.

1.5 Centralized and decentralized control

When the process is made of many parts — the intersections of a city, the buildings of a district, the vehicles of a fleet — one can write a single optimization problem for the whole system, which is *centralized* MPC, or one problem per part, each with its own controller, which is *decentralized* MPC; when the local controllers exchange information, the scheme is called *distributed*. The centralized problem is optimal but large; the local problems are small but have to deal with the *coupling constraints* that tie the parts together. The case studies of this part are centralized; the question comes back with the aggregation of electric vehicles and with the control of several intersections.

1.6 Hard and soft constraints

A constraint is *hard* when it must hold without exception: the physical limit of an actuator, the safety limit of a tank. It is *soft* when it expresses a preference that may be violated at a cost: a reference level, a deadline for a job. Soft constraints are written by adding a non-negative

slack variable that measures the violation and by charging it in the cost; section 7.7 shows the construction. The distinction matters because a problem whose hard constraints cannot all be satisfied has no solution, and a controller with no solution has nothing to apply.

Further reading

[Rawlings_2017], sections 1.1 and 1.2 (pp. 1–10), for the setting of the problem and for modelling; [Cannon_2016], chapter 1, for an overview of predictive control. The slides on MPC of the course website, up to slide 25, cover this chapter.

2 Models and the optimization problem of MPC

2.1 State-space models

A *dynamical system* is described by a set of *state variables*, collected in the state vector $x \in \mathbb{R}^n$, which summarize its past: given the state at one instant and the inputs from then on, the future of the system is determined. The inputs are collected in $u \in \mathbb{R}^m$, the measured outputs in $y \in \mathbb{R}^p$.

In continuous time the model is a differential equation, $\dot{x}(t) = f_c(x(t), u(t))$. The controller, however, acts at the sampling instants, so the model it uses is in discrete time:

$$x_{k+1} = f(x_k, u_k), \qquad y_k = h(x_k). \tag{1}$$

The model is *linear* when $f(x, u) = Ax + Bu$ and $h(x) = Cx$ for constant matrices A, B, C; it is *nonlinear* otherwise. Three examples were worked out in class: a mass connected to a spring and a damper and pushed by a force, whose state is the pair (position, velocity) and whose model is linear; a pendulum driven by a torque, nonlinear because of the gravitational term $\sin\theta$; an RLC circuit, linear. The procedure is always the same: write the physical balance equations, choose the states, the inputs and the outputs, and rewrite the equations in the form (1).

An *equilibrium* of (1) under a constant input $\bar{u}$ is a state x^{eq} that the system does not leave, $x^{eq} = f(x^{eq}, \bar{u})$. A system may have one, several or infinitely many equilibria, or none: the pendulum has two families of them, the hanging and the upright position repeated at every full turn. Without loss of generality the equilibrium of interest is placed at the origin, $f(0, 0) = 0$, by a change of variables.

2.2 The problem solved at every step

The optimization problem that MPC solves at time k, from the measured state x_k, is

$$\begin{aligned}
\mathcal{P}_N(x_k): \quad \min_{u_{0|k},\dots,u_{N-1|k}} \quad & V_N(x_k, \mathbf{u}_k) = \sum_{i=0}^{N-1} \ell(x_{i|k}, u_{i|k}) + V_f(x_{N|k}) \\
\text{s.t.} \quad & x_{i+1|k} = f(x_{i|k}, u_{i|k}), \qquad i = 0, \dots, N-1, \\
& x_{0|k} = x_k, \\
& u_{i|k} \in \mathcal{U}, \qquad i = 0, \dots, N-1, \\
& x_{i|k} \in \mathcal{X}, \qquad i = 0, \dots, N, \\
& x_{N|k} \in \mathcal{X}_f.
\end{aligned} \tag{2}$$

Its ingredients are the following.

- The *horizon* N: the number of steps the controller looks ahead. A longer horizon gives a better plan at the price of a larger problem.

- The *stage cost* $\ell(x, u)$: the cost paid at each step, which encodes the goal. For a regulation problem, where the state has to be brought to the origin, the usual choice is quadratic,

$$\ell(x, u) = x^\top Q x + u^\top R u, \tag{3}$$

  with R positive definite and Q positive definite, or positive semi-definite when only some of the states are penalized.

- The *terminal cost* $V_f(x)$: the cost charged on the last predicted state. Its role, which is central, is the subject of section 3.

- The *dynamics* as equality constraints, with the measured state as initial condition: this is where the model enters, and what makes the controller a feedback.

- The sets $\mathcal{U}$ and $\mathcal{X}$ of admissible inputs and states: the limits of the process. The common case is a box, $u^{\min} \le u \le u^{\max}$, $x^{\min} \le x \le x^{\max}$.

- The *terminal set* $\mathcal{X}_f$, a constraint on the last predicted state. It is not required by the application: it is added to guarantee that the problem remains solvable from one step to the next, as section 4 explains.

The optimal value of the problem is written $V_N^*(x_k)$, and the first element of the optimal sequence defines the control law, $\kappa_N(x_k) = u_{0|k}^*$.

Example 2.1 (the tank)**.** A tank of water volume x is filled by a pump of power u and emptied by a known consumption w. With a sampling time T, the volume evolves as $x_{k+1} = x_k + a\,u_k - b\,w_k$. The volume has to stay between a minimum and a maximum, the pump between zero and its rated power, and the level has to be kept close to a reference x^{ref}. The problem solved at every step is (2) with $\ell(x, u) = (x - x^{ref})^2 + \rho\,u^2$, box constraints on x and u, and no terminal set. Because the model is linear and the cost quadratic, $\mathcal{P}_N(x_k)$ is a quadratic program. The Tank Control Lab on the course website runs this controller next to a PID on the same tank: the PID has no knowledge of the limits and of the future consumption, and the difference is visible in the volume trajectory and in the number of minutes spent outside the admissible band.

2.3 The ideal problem: infinite horizon

Behind (2) there is the problem one would really like to solve, the *infinite-horizon constrained optimal control problem*:

$$\min_{u_0, u_1, \dots} \sum_{i=0}^{\infty} \ell(x_i, u_i) \quad \text{s.t.} \quad x_{i+1} = f(x_i, u_i),\ x_0 = x_k,\ u_i \in \mathcal{U},\ x_i \in \mathcal{X}. \tag{4}$$

It has infinitely many variables and constraints, and in general it cannot be solved. With a linear model and the quadratic cost (3) it is the *constrained LQR* problem, still out of reach because of the constraints.

The one case that can be solved exactly is the *unconstrained* linear-quadratic problem. Its solution is a linear state feedback, $u_k^* = K x_k$, with the gain K given by the discrete-time Riccati equation, and the closed-loop cost from any initial state is a quadratic form:

$$J^*(x_0) = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_k^\top R u_k\right) = x_0^\top W x_0, \tag{5}$$

where W solves the Lyapunov equation of the closed loop. The derivation is short: under the law $u = Kx$ the state is $x_k = (A + BK)^k x_0$, the input $u_k = K(A + BK)^k x_0$, and substituting both in the sum collapses it into a single quadratic form in x_0. The matrix W is the quantity that does all the work in the next chapter.

Further reading

[Cannon_2016], section 2.1, "Problem Description" (pp. 13–15): the same problem and the same notation as in these notes; section 2.2, "The Unconstrained Optimum" (pp. 15–18): the cost under a linear feedback, the Lyapunov equation for W and the Riccati equation for the gain. [Rawlings_2017], section 2.2, "Model Predictive Control" (pp. 91–106), for the general formulation with stage cost, terminal cost and terminal set, and section 1.2, "Models and Modeling" (pp. 1–10), for the modelling part.

3 Optimality: a finite window for an infinite problem

This chapter answers one question: in what sense is the finite-horizon problem $\mathcal{P}_N(x_k)$ a substitute for the infinite-horizon problem (4)? The answer goes through the value function and the principle of optimality, and it identifies the terminal cost V_f as the ingredient that makes the substitution exact. Throughout the chapter three assumptions hold: the model is known and perfect, the state is measured, there are no disturbances.

3.1 The value function and Bellman's principle

Definition 3.1 (value function)**.** The *value function* $V(x)$ is the optimal cost of the infinite-horizon problem (4) started at x: the cost of controlling the system optimally from x onwards. It is $+\infty$ when no admissible input sequence exists from x.

Theorem 3.2 (principle of optimality)**.** *Let $\{u_0^*, u_1^*, \dots\}$ be an optimal input sequence for the problem started at x_0, and let x_N^* be the state it reaches at time N. Then the tail $\{u_N^*, u_{N+1}^*, \dots\}$ is an optimal input sequence for the problem started at x_N^*.*

The reason is simple: if a cheaper tail existed from x_N^*, replacing the tail of the original sequence with it would give a cheaper sequence from x_0, against the optimality of the original. The consequence is a decomposition of the optimal cost at any junction time N:

$$V(x_0) = \sum_{i=0}^{N-1} \ell(x_i^*, u_i^*) + V(x_N^*). \tag{6}$$

The optimal cost from x_0 is the cost of the first N steps plus the value function of the state reached: the infinite tail is summarized by one number.

3.2 Dual-mode control

Consider the constrained linear-quadratic problem. Assume the system controllable and the origin strictly inside $\mathcal{X}$ and $\mathcal{U}$. Then an optimal sequence exists and its cost is finite; a finite sum of positive-definite terms forces $x_k \to 0$ and $u_k \to 0$, so the constraints can be active only in an initial phase. After some finite time N the state is close enough to the origin that the trajectory generated by the unconstrained law $u = Kx$ respects all the constraints from then on, and by the principle of optimality applied to the tail problem that law is not only admissible there, it is *optimal*, because from that point on the constrained and the unconstrained problems have the same solution.

This is *dual-mode control*: the input is free in the first N steps, and it is the LQR feedback afterwards (fig. 2). The tail is then known in closed form, and so is its cost, $x_N^\top W x_N$ from (5). Substituting it in (6), the infinite problem becomes

$$\min_{u_0,\dots,u_{N-1}} \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i) + x_N^\top W x_N, \tag{7}$$

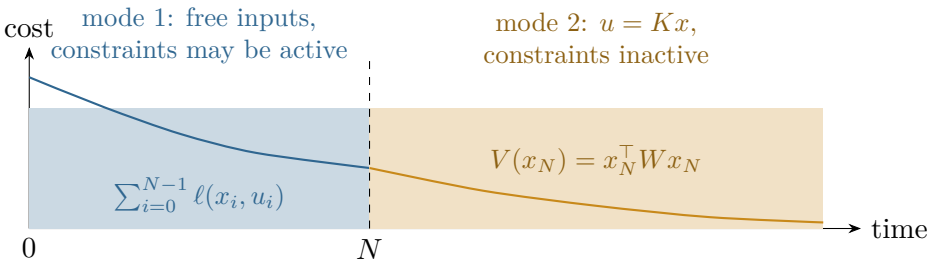


Figure 2: Dual-mode control. In the first N steps the inputs are free and the constraints may be active; from N on the unconstrained LQR feedback is optimal, and the cost of that tail is the quadratic form $x_N^\top W x_N$, which becomes the terminal cost of a finite-horizon problem.

subject to the dynamics and the constraints over the first N steps. This problem has N decision variables and it is solvable. Its solution coincides with the first N inputs of the infinite-horizon optimum, $u_k^{*N} = u_k^{*\infty}$ for $k = 0, \dots, N-1$, provided N is large enough for the tail to be unconstrained.

3.3 The terminal cost as the cost-to-go

Problem (7) is $\mathcal{P}_N(x_k)$ with the quadratic stage cost and with terminal cost $V_f(x) = x^\top W x$. The lesson of the construction is general: the terminal cost of an MPC problem should be the *value function* of the infinite-horizon problem, the cost of what happens after the window. When it is, the finite problem and the infinite one have the same solution, and solving the finite problem again at every sampling time — which is MPC — reproduces the infinite-horizon optimum. This is the precise sense in which MPC is optimal.

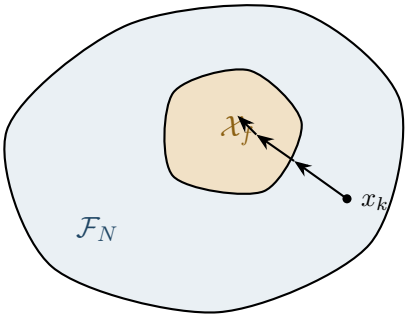
The practical difficulty is that $V(x)$ is unknown in general. The unconstrained linear-quadratic problem is the exception: there $V(x) = x^\top W x$ exactly, and this is why, in linear-quadratic MPC, the terminal cost is taken from the Riccati or the Lyapunov equation and the horizon is taken long enough. In other problems $V(x)$ has to be approximated, or learned: the value function is also the object computed by dynamic programming, and learning it from experience is the subject of reinforcement learning, the second part of the course.

Remark 3.3 (the terminal weight matters). The Service Area Lab on the website shows what happens when the terminal cost is not the value function. With a window of twenty minutes and a terminal weight equal to the stage weight on the stored energy, the controller is myopic: it sees almost no value in the state it leaves at the end of the window, and the storage drifts away from its reference. With the terminal weight from the Riccati equation, about four hundred times larger, the same twenty-minute controller behaves like the infinite-horizon one.

3.4 From one step to the next

Under the nominal assumptions, re-solving the problem at time $k+1$ returns the plan the controller already had: $u_{1|k}^*$ becomes $u_{0|k+1}^*$, and so on. The controller confirms itself, and the receding horizon changes nothing. In reality the model is not perfect, the forecasts are wrong and disturbances act on the process, so the state measured at $k+1$ is not the one predicted; this is why the plan is rebuilt at every step rather than computed once.

The cost actually paid by the controller in closed loop is the sum of the stage costs along the trajectory it generates, $J^{cl}(x_0) = \sum_{k \geq 0} \ell(x_k, \kappa_N(x_k))$. Its difference from $V(x_0)$ is the *optimality gap*; it is zero in the ideal conditions of this chapter, and it measures what is lost when they fail.



every arrow stays inside:
the set is controlled invariant

Figure 3: The feasible set $\mathcal{F}_N$ and the terminal set $\mathcal{X}_f$. Recursive feasibility means that the control law never takes the state out of $\mathcal{F}_N$; the terminal set is the smaller set, controlled invariant under the terminal law, that makes the proof work.

Further reading

[Cannon_2016], section 2.3, "The Dual-Mode Prediction Paradigm" (pp. 18–21); Theorem 2.2 there is the formal version of the result of this chapter. [Rawlings_2017], sections 1.3.3 and 1.3.4 (pp. 18–23) and section 2.3, "Dynamic Programming Solution" (pp. 107–112), for the same material reached through dynamic programming, which is the road taken again in the reinforcement learning part. Studying the proofs is optional.

4 Recursive feasibility

A controller that applies the first step of a plan needs a plan at every step. If the optimization problem has no admissible solution at some time k, the controller has nothing to apply. The property that rules this out is the subject of this chapter.

4.1 Feasible sets and invariance

Definition 4.1 (feasible set)**.** The *feasible set* $\mathcal{F}_N$ of problem (2) is the set of states x from which the problem admits at least one admissible input sequence.

Definition 4.2 (recursive feasibility)**.** The MPC algorithm is *recursively feasible* if, whenever $\mathcal{P}_N(x_k)$ has a solution, $\mathcal{P}_N(x_{k+1})$ has a solution as well, with $x_{k+1} = f(x_k, \kappa_N(x_k))$. In words: feasible once, feasible forever.

Recursive feasibility is a statement about sets. For an autonomous system $x_{k+1} = g(x_k)$ a set S is *positively invariant* when every state in S is mapped into S: $x \in S \Rightarrow g(x) \in S$. For a controlled system $x_{k+1} = f(x_k, u_k)$ a set S is *controlled invariant* when for every $x \in S$ there exists an admissible input $u \in \mathcal{U}$ such that $f(x, u) \in S$. Recursive feasibility is exactly the statement that the MPC law makes the feasible set controlled invariant: from every state in $\mathcal{F}_N$, the input it chooses leads to a state that is again in $\mathcal{F}_N$ (fig. 3).

4.2 The proof

The proof is constructive, and the construction is the natural one. Assume the nominal conditions, so that the state at $k+1$ is exactly the one the controller predicted, $x_{k+1} = x^*_{1|k}$. At time k the controller found the optimal sequence $\{u^*_{0|k}, u^*_{1|k}, \dots, u^*_{N-1|k}\}$ with predicted states $\{x^*_{0|k}, \dots, x^*_{N|k}\}$, and $x^*_{N|k} \in \mathcal{X}_f$. At time $k+1$ build the *tentative* sequence

$$\tilde{\mathbf{u}}_{k+1} = \{\, u^*_{1|k},\, u^*_{2|k},\, \dots,\, u^*_{N-1|k},\, \kappa_f(x^*_{N|k}) \,\}, \tag{8}$$

the tail of the previous optimal sequence, completed with one more input given by a *terminal control law* κ_f applied to the previous terminal state. The first $N-1$ inputs are admissible and produce admissible states, because they did so at time k — the window has merely moved. Two things remain to be checked: that the appended input is admissible, $\kappa_f(x^*_{N|k}) \in \mathcal{U}$, and that the state it produces satisfies the terminal constraint, $f(x^*_{N|k}, \kappa_f(x^*_{N|k})) \in \mathcal{X}_f$.

Both hold at once if the terminal set and the terminal law are chosen so that

$$x \in \mathcal{X}_f \;\Rightarrow\; \kappa_f(x) \in \mathcal{U} \;\text{ and }\; f(x, \kappa_f(x)) \in \mathcal{X}_f, \tag{9}$$

that is, if κ_f makes $\mathcal{X}_f$ positively invariant for the controlled system while respecting the input constraints. Then (8) is an admissible sequence for $\mathcal{P}_N(x_{k+1})$, the problem has at least one solution, and recursive feasibility follows.

Theorem 4.3 (recursive feasibility)**.** *Under the nominal assumptions, if $\mathcal{X}_f$ and κ_f satisfy* (9)*, then $\mathcal{P}_N(x_k)$ feasible implies $\mathcal{P}_N(x_{k+1})$ feasible, and the feasible set $\mathcal{F}_N$ is controlled invariant under the MPC law.*

4.3 Choosing the terminal set

Two examples of terminal law appeared in class. The law that does nothing, $\kappa_f(x) = 0$, works when $\mathcal{X}_f$ is invariant for the uncontrolled system. The linear law $\kappa_f(x) = Kx$, with K the LQR gain, is the one that connects back to dual-mode control: $\mathcal{X}_f$ is then a set inside which the unconstrained feedback respects all the constraints, and the second mode of fig. 2 is exactly the terminal law.

The choice involves a trade-off. The terminal constraint $x_{N|k} \in \mathcal{X}_f$ is *added* to the problem; it is not required by the application, and like every constraint it can only make the optimal cost worse. A larger terminal set therefore means a larger feasible set and a lower cost, and one looks for the largest $\mathcal{X}_f$ that admits a law satisfying (9). Under that condition the feasible sets also grow with the horizon: $\mathcal{F}_N \subseteq \mathcal{F}_{N+1}$, because a sequence that reaches $\mathcal{X}_f$ in N steps, extended with one step of the terminal law, reaches it in $N+1$.

Remark 4.4 (when feasibility fails). The Building Energy Lab on the website makes the failure visible: lower the contractual power limit while forcing several appliances into the same narrow window, and the optimizer reports that no admissible plan exists. The constraints are contradictory; the algorithm is not at fault. The remedy, when the application allows it, is to make the constraints that may fail soft, as in section 7.7.

Further reading

[Cannon_2016], section 2.4, “Invariant Sets” (pp. 21–24), and section 2.5, “Controlled Invariant Sets and Recursive Feasibility” (pp. 24–29): the proof above in the same form, with the nested feasible sets. [Rawlings_2017], section 2.4.2, “Stabilizing Conditions” (pp. 114–120), for the assumptions on the terminal set in their standard form, and section 2.6, “Is a Terminal Constraint Set Necessary?” (pp. 144–147), for what happens without one.

5 Stability

Stability is a property of the equilibrium points of a dynamical system, not of the system as a whole. Adding a feedback law turns the controlled system $x_{k+1} = f(x_k, u_k)$ into the autonomous system $x_{k+1} = f(x_k, \kappa(x_k)) = \tilde{f}(x_k)$, and two questions arise: can an unstable equilibrium be made stable by the controller, and can an equilibrium that was stable be made unstable by it? For MPC the second question is the one that needs an answer, because κ_N is a nonlinear law that no classical criterion covers.

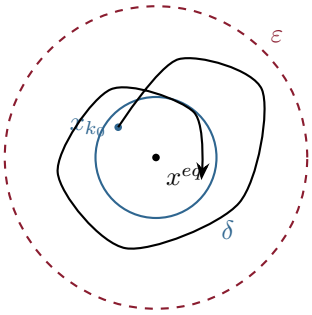


Figure 4: The ε–δ definition of stability: a trajectory that starts within δ of the equilibrium never leaves the ball of radius ε.

5.1 Definitions

Consider an autonomous system $x_{k+1} = \tilde{f}(x_k)$ with an equilibrium at the origin, $\tilde{f}(0) = 0$.

Definition 5.1 (stability)**.** The origin is *stable* if for every $\varepsilon > 0$ there exists $\delta(\varepsilon) > 0$ such that $\|x_{k_0}\| \leq \delta$ implies $\|x_k\| \leq \varepsilon$ for all $k \geq k_0$. It is *asymptotically stable* if it is stable and, in addition, $x_k \to 0$ for every initial state in some neighbourhood of the origin, the *region of attraction*. It is *globally asymptotically stable* if the region of attraction is the whole state space.

Stability says that small initial perturbations produce trajectories that remain forever close (fig. 4); asymptotic stability adds that they return. Both are local notions unless stated otherwise.

5.2 Linear systems

For $x_{k+1} = Ax_k$ the answer is complete. The solution is $x_k = A^k x_0$, and everything depends on the eigenvalues of A: the origin is asymptotically stable if and only if all the eigenvalues lie strictly inside the unit circle; it is unstable if at least one lies outside; eigenvalues on the circle give simple stability when they are simple, and need a separate check when they are repeated. This criterion is not enough for MPC: the model may be nonlinear, and even for a linear model the controller κ_N is nonlinear.

5.3 Lyapunov functions

Definition 5.2 (definite functions)**.** A scalar function of the state $V : \mathbb{R}^n \to \mathbb{R}$ is *positive definite* if $V(0) = 0$ and $V(x) > 0$ for every $x \neq 0$; it is *positive semi-definite* if $V(0) = 0$ and $V(x) \geq 0$ for every x. It is negative definite, or semi-definite, when $-V$ is positive definite, or semi-definite. The *local* versions of these definitions require the inequalities only in a neighbourhood of the equilibrium.

The quadratic forms $V(x) = x^\top P x$ are the standard example: such a form is positive definite if and only if all the eigenvalues of P are positive. Not every candidate works: $V(x) = 2\sin^2(x_1) + 3x_2^2$ is positive definite only locally, since it vanishes at $(\pi, 0)$. The level sets $\Omega_c = \{x : V(x) = c\}$ of a positive definite function are closed curves around the origin, nested as c grows; they are the geometric picture behind the theorems that follow.

The *increment* of V along the trajectories of the system is

$$\Delta V(x) = V\big(\tilde{f}(x)\big) - V(x), \tag{10}$$

the change of V from one state to the next. It is computed along the trajectories, not in an arbitrary direction.

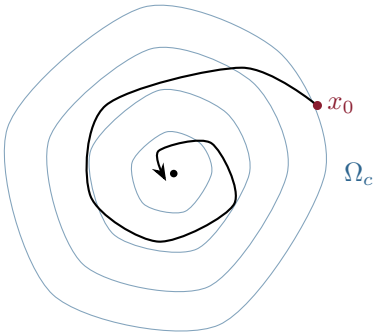


Figure 5: Level sets of a positive definite function and a trajectory along which V decreases at every step: each level set traps the trajectory inside a smaller one.

Theorem 5.3 (Lyapunov)**.** *Let the origin be an equilibrium of $x_{k+1} = \tilde{f}(x_k)$. If there exists a continuous function V, positive definite at least locally around the origin, such that ΔV is negative semi-definite at least locally, then the origin is stable. If, in addition, ΔV is negative definite, then the origin is asymptotically stable.*

The condition $\Delta V \leq 0$ is often written as $\Delta V(x) \leq -h(x)$ with $h \geq 0$, which says *how much* V must decrease at each step; this form is the one used in the next section. The intuition is in fig. 5: if V is positive and decreases along the trajectory, the trajectory is confined to level sets of smaller and smaller value, and if the decrease is strict then $V \to 0$ and with it $x_k \to 0$. A function tried as a candidate is a *candidate Lyapunov function*; one that satisfies both conditions is a *Lyapunov function*. The theorem is a sufficient condition: failing to find a Lyapunov function proves nothing.

5.4 Stability of MPC

Take the regulation problem: the system has an equilibrium at the origin, $f(0,0) = 0$, the stage cost $\ell(x,u)$ is positive definite in (x,u), and the terminal cost $V_f(x)$ is positive definite in x. These assumptions are satisfied by any reasonable regulation cost, such as (3) with Q and R positive definite.

The equilibrium survives. The closed loop is $x_{k+1} = f(x_k, \kappa_N(x_k)) = \tilde{f}(x_k)$, so the origin is an equilibrium of $\tilde{f}$ if and only if $\kappa_N(0) = 0$. It is: from $x_k = 0$ the sequence of all zeros is admissible, keeps the state at zero and costs nothing, and since the stage and terminal costs are positive definite no other sequence can cost less. The optimizer returns zero, and the controller does not move the equilibrium.

The candidate. The candidate Lyapunov function is the *optimal cost of the MPC problem*, $V_N^*(x_k) = V_N(x_k, \mathbf{u}_k^*)$: a function of the state alone, computed by the controller at every step in any case. It is positive definite: $V_N^*(0) = 0$ by the argument above, and for $x \neq 0$ the first stage cost $\ell(x_k, u_{0|k}^*)$ alone is strictly positive.

The decrease. We need $V_N^*(x_{k+1}) - V_N^*(x_k) \leq 0$, and $V_N^*(x_{k+1})$ is the result of an optimization that we have not performed. The way around it is a bound: the optimal cost at $k+1$ is never larger than the cost of *any* admissible sequence at $k+1$, so it is enough to exhibit one admissible sequence whose cost is already below $V_N^*(x_k)$. The candidate is the one of section 4: the tail of the optimal sequence of time k, completed with the terminal control,

which is admissible under the nominal assumption $x_{k+1} = x^*_{1|k}$ and condition (9). Writing both costs term by term, the stage costs from 1 to $N-1$ cancel and what is left is

$$V_N(x_{k+1}, \tilde{\mathbf{u}}_{k+1}) - V^*_N(x_k) = \underbrace{-\ell(x_k, u^*_{0|k})}_{\leq 0} + \Big[\ell(x^*_{N|k}, \kappa_f(x^*_{N|k})) + V_f\big(f(x^*_{N|k}, \kappa_f(x^*_{N|k}))\big) - V_f(x^*_{N|k})\Big]. \tag{11}$$

The first term is the cost paid at the current step and is never positive. The whole difference is therefore non-positive as soon as the bracket is, and the bracket involves the terminal ingredients only.

Theorem 5.4 (stability of MPC)**.** *Let $f(0,0) = 0$, let ℓ and V_f be positive definite, let $\mathcal{X}_f$ and κ_f satisfy* (9)*, and let*

$$V_f\big(f(x, \kappa_f(x))\big) - V_f(x) \;\leq\; -\ell(x, \kappa_f(x)) \qquad \text{for every } x \in \mathcal{X}_f. \tag{12}$$

Then V^*_N is a Lyapunov function for the closed loop, $\Delta V^*_N(x_k) \leq -\ell(x_k, \kappa_N(x_k))$, and the origin is asymptotically stable with the feasible set as region of attraction.

Condition (12) says that the terminal cost must itself be a Lyapunov function for the system driven by the terminal law, decreasing by at least the stage cost at every step. Stability of MPC is therefore not something one hopes for: it is obtained by choosing V_f and κ_f properly. The practical reading is the following. If a feedback law $u = c(x)$ is known to stabilize the origin, with a Lyapunov function W proving it and a decrease of at least the stage cost, then taking $V_f \equiv W$ and $\kappa_f \equiv c$ makes the MPC controller stabilizing: the proof above becomes exactly the property of W. For the linear-quadratic problem this is the LQR law with $V_f(x) = x^\top W x$, the same choice that section 3 obtained for optimality.

5.5 The role of each ingredient

The theory of the last three chapters assigns a role to each ingredient of $\mathcal{P}_N(x_k)$.

- The *horizon* N trades optimality for computation: as N grows the problem approaches the infinite-horizon one, which is the best that can be done and which cannot be solved; at the same time the number of variables and constraints grows, and so does the solution time.
- The *terminal cost* V_f serves optimality, when it is the value function of the infinite-horizon problem, and stability, when it satisfies (12). When the value function is not available it can be approximated, or learned.
- The *terminal set* $\mathcal{X}_f$ serves recursive feasibility, through (9). It is an artificial constraint, added for the proof and not by the application, and it costs performance: the larger it can be made, the better, the limit being $\mathcal{X}_f = \mathcal{X}$ when $\mathcal{X}$ itself satisfies (9).

Further reading

[Cannon_2016], section 2.6, "Stability and Convergence" (pp. 29–32): the proof of this chapter in the same form, with the optimal cost as Lyapunov function and the condition on the terminal cost. [Rawlings_2017], appendix B, section B.2, "Stability and Asymptotic Stability" (pp. 696–700), for the definitions, and section B.3, "Lyapunov Stability Theory" (pp. 700–709), for the Lyapunov theorems in their general form; section 2.4.2, "Stabilizing Conditions" (pp. 114–120), for the assumptions on ℓ, V_f, κ_f and $\mathcal{X}_f$ as they are used

throughout that book. [Mayne_2000] is the survey in which these conditions were first collected in the form used today.

6 Economic MPC and the design of the cost

6.1 Tracking and economic objectives

The theory of the previous chapters was developed for *regulation* and *tracking* problems: the control goals are first translated into reference values for the state or the output, and the cost penalizes the distance from those references. Such a cost is positive definite by construction, which is what the stability proof requires.

In many applications the goal is not a reference but a quantity with a direct meaning, usually money: the electricity bill of a building, the cost of running a plant, the revenue of a service. *Economic MPC* writes that quantity directly as the cost, solves the problem, and applies the first input. Nothing is tracked. The formulation is more natural, and the price is that the assumptions of the theory are properties of the cost that an economic cost need not have: a bill is not positive definite in the state, and the stability and optimality results of section 5 do not apply as they stand. The case studies of section 8 are economic MPC problems; their properties are studied, in the literature, with tools beyond this course.

6.2 Several goals: the weighted sum

A real controller has several goals at once: a bill to minimize, a storage to keep near a reference, a service to deliver, a device to spare. The standard way to handle them is to write one term per goal and to take a *weighted sum*,

$$J = \alpha_1 J_1 + \alpha_2 J_2 + \cdots + \alpha_r J_r, \qquad \alpha_j \geq 0. \tag{13}$$

Multiplying all the weights by the same positive constant does not change the minimizer, so the weights can be normalized to sum to one, $\sum_j \alpha_j = 1$: only their ratios matter.

The set of cost pairs (J_1, J_2) that admissible decisions can achieve, for two goals, is sketched in fig. 6. Its lower-left boundary is the *Pareto frontier*: the solutions that cannot be improved in one goal without worsening the other. Minimizing the weighted sum (13) with a given pair of weights selects one point of the frontier (this is the *scalarization* of a vector optimization problem); sweeping the weights traces the frontier — all of it when the problem is convex, as in fig. 6, and only its convex part otherwise. A rule-based controller, not being optimal, sits above the frontier; a better algorithm moves the whole frontier towards the origin. Choosing the weights is therefore a design decision about trade-offs, taken by whoever operates the process, not a mathematical one — and it is a decision that can be learned from data, which is one of the points where reinforcement learning enters the picture.

6.3 The terminal value of a stored resource

Economic problems have their own version of the terminal cost, and the building case study (section 8.2) shows it. A controller that minimizes a bill over a window of one day sells the energy stored in a battery before the window ends, because within the window that sale is a profit and what happens afterwards does not exist. The behaviour is correct for the problem written, and wrong for the application.

The remedy is to maximize the *total value* over the window: the value of the state the controller starts from, plus the profits minus the costs of what it plans, plus the value of the

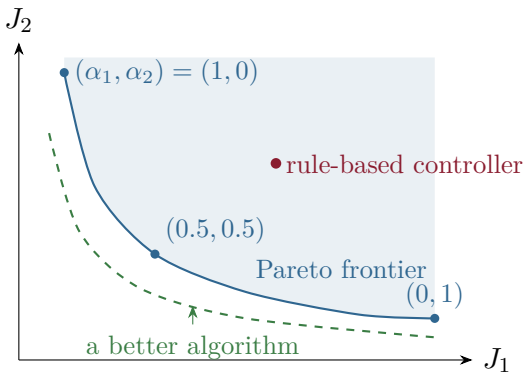


Figure 6: Two goals to minimize. The weighted sum with a given pair of weights selects one point of the Pareto frontier; a rule-based controller lies above it; an algorithm with a larger decision space pushes the frontier down.

state it leaves behind. The first term does not depend on the decisions, so maximizing the total value is the same as minimizing

$$J^{cost} - V(x_{N|k}), \tag{14}$$

where $V(x_{N|k})$ is the price put on the resource still stored at the end of the window — for a battery, $V(x) = c^{avg}\, x$ with c^{avg} the average purchase price of energy over the day. This is the terminal cost of section 3 seen from the side of the application: it prices what happens after the horizon, and without it the controller is myopic in exactly the way the theory predicts.

Further reading

[Rawlings_2017], section 2.8, "Economic Model Predictive Control" (pp. 153–160): how the economic problem differs from the tracking one, and which assumptions replace the positive definite cost. [Boyd_2004], section 4.7, "Vector optimization" (pp. 174–187), in particular 4.7.3 on Pareto optimal points and 4.7.4 on scalarization, which is the weighted sum (13) and the reason the weights can be normalized.

7 Writing logic as constraints

Many decisions in the case studies are logical: a storage charges or discharges, an appliance starts or does not, a light is green or red, a task runs on a machine or not. Optimization solvers do not handle logic; they handle linear inequalities. This chapter collects the constructions that translate the one into the other. Each uses *binary variables*, $\delta \in \{0, 1\}$, and the result is a mixed-integer linear program — harder than a continuous one, but far more tractable than a nonlinear one. The constructions are used repeatedly in section 8.

7.1 Indicator variables and the big-M constraint

Let $p \geq 0$ be a continuous variable with a known upper bound $p^{\max}$, and let δ be the binary that is meant to indicate whether p is positive. The *indicator constraint*

$$p \leq M\,\delta, \qquad M \geq p^{\max}, \tag{15}$$

forces $p = 0$ when $\delta = 0$ and leaves p free in $[0, M]$ when $\delta = 1$. It does not force $\delta = 1$ whenever $p > 0$ by itself; that direction is not needed when the cost makes an unnecessary $\delta = 1$ never cheaper, which is the usual situation. The constant M should be as small as the

bounds allow, $M = p^{\max}$, so that the relaxation of the problem — the one obtained by letting δ take values in $[0, 1]$, which solvers use to guide the search — is as tight as possible.

7.2 Mutually exclusive events

Two events that cannot happen at the same time — charging and discharging a battery, importing and exporting power at a meter — are written with one indicator each and the constraint

$$\delta^a + \delta^b \le 1. \tag{16}$$

Combined with (15) on each power, at most one of the two powers can be positive at a time. The construction is what makes it possible to write a quantity with two different models in the two cases, for instance a storage with a charging efficiency and a different discharging efficiency:

$$\begin{aligned} p^{ess} &= p^{ch} - p^{dis}, \qquad & x_{k+1} &= x_k + \eta^{ch}\, T\, p^{ch}_k - \tfrac{1}{\eta^{dis}}\, T\, p^{dis}_k, \\ p^{ch} &\le p^{ch,\max}\delta^{ch}, \qquad & p^{dis} &\le p^{dis,\max}\delta^{dis}, \qquad & \delta^{ch} + \delta^{dis} &\le 1, \end{aligned} \tag{17}$$

which is linear, whereas the same model written with $\max\{0, p\}$ and $\min\{0, p\}$ is not.

7.3 Semi-continuous variables

A device that is either off or working between a minimum and a maximum level — a charger that cannot deliver an arbitrarily small power — has a *semi-continuous* variable: $p = 0$ or $p^{\min} \le p \le p^{\max}$. With one indicator the two inequalities

$$\delta\, p^{\min} \;\le\; p \;\le\; \delta\, p^{\max} \tag{18}$$

force $p = 0$ when $\delta = 0$ and $p \in [p^{\min}, p^{\max}]$ when $\delta = 1$.

7.4 One start per job: one-hot variables

A job with a known duration and a known power profile has to be executed once, inside a window between an earliest start S and a deadline F. The decision is *when* to start it, and it is written with one binary per admissible start time, $s_j \in \{0, 1\}$ for $j = S, \dots, F - N_{job}$, where N_{job} is the duration in steps and the upper limit is the last instant at which the job still finishes before the deadline. Exactly one of them is one:

$$\sum_{j=S}^{F-N_{job}} s_j = 1. \tag{19}$$

The power of the job at time i is then the profile placed where the start variable says,

$$p_i = \sum_j s_j\, P(i - j + 1), \tag{20}$$

a convolution of the profile $P(\cdot)$ with the one-hot sequence (fig. 7).

7.5 The product of a real variable and a binary

A term $q\, b$, with $q \in [\underline{q}, \bar{q}]$ real and $b \in \{0, 1\}$, is not linear. It is replaced by a new variable p and two pairs of inequalities:

$$\underline{q}\, b \le p \le \bar{q}\, b, \qquad q - M(1 - b) \le p \le q + M(1 - b), \qquad M \ge \max\{\left|\underline{q}\right|, |\bar{q}|\}. \tag{21}$$

When $b = 0$ the first pair forces $p = 0$ and the second says nothing; when $b = 1$ the second pair forces $p = q$ and the first is vacuous. The construction recurs in every formulation with timers, such as the duration of a traffic-light colour, $t_{i+1} = (t_i + T)\, y_i$.

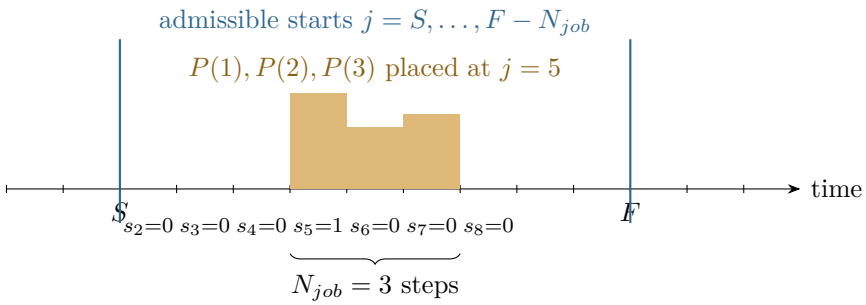


Figure 7: Load shifting with one-hot start variables. Exactly one s_j equals one inside the admissible range, and the power profile of the job is placed at that start.

7.6 Maxima and absolute values

A goal of the form "minimize the largest of several quantities" is not a sum. It is handled with an *epigraph* variable: to minimize $\max_v w_v$ over a set of quantities w_v, introduce $\bar{w}$ with the constraints $w_v \le \bar{w}$ for every v, and minimize $\bar{w}$. The bound is pushed down until it rests on the largest w_v. The same device writes an absolute value in a cost: $|x|$ is replaced by $x^+ + x^-$ with $x = x^+ - x^-$, $x^+, x^- \ge 0$, exact as long as the cost is increasing in both; this is how a tracking error $\left|P - P^{ref}\right|$ enters a linear program. A convex piecewise-linear cost, such as an approximation of a square, is written in the same way with several pieces of increasing price.

7.7 Soft constraints and slack variables

A constraint that may be impossible to satisfy — a deadline for a charge when the vehicle arrives too late, a stop requested in the only hours in which a load can run — must not be written as a hard constraint, or the problem becomes infeasible and the controller has nothing to apply. It is *relaxed* with a non-negative slack variable that measures the violation, and the violation is paid for in the cost:

$$g(x) \le 0 \quad \longrightarrow \quad g(x) \le \varepsilon, \quad \varepsilon \ge 0, \qquad J \leftarrow J + d\,\varepsilon \;\text{ or }\; J + d\,\varepsilon^2. \tag{22}$$

The weight d sets the behaviour: small, and the controller violates the constraint when that improves the other goals; large, and it violates it only when the constraint cannot be met at all. With several relaxed constraints the quadratic form $\sum_p \varepsilon_p^2$ is preferable to the linear one, because it prefers many small violations to one large one — it is fairer.

Further reading

[Hax_1977], chapter 9, for integer programming and the modelling of logical conditions: indicator variables, big-M, mutually exclusive alternatives, fixed charges. [Borrelli_2017] for the same constructions inside an MPC problem, in the framework of mixed logical dynamical systems. [Boyd_2004], section 4.1.3, for the epigraph form of an optimization problem.

8 Case studies

The case studies follow the same pattern. The control goals are identified; they are turned into a cost; the system is modelled with states, inputs and disturbances; the limits of the system and the logic of the application become constraints; and the resulting problem is solved at every step on a receding horizon. All of them are economic MPC problems in the sense of

section 6, and all use the constructions of section 7. For each of them the course website has an interactive laboratory that runs the controller in the browser.

8.1 The charging service area

Problem. A service area for the fast charging of electric vehicles draws power from the grid at a point of connection (POC). Vehicles arrive wanting to recharge as fast as possible, so even a few of them produce a large power flow, which means a large connection and a heavy bill under a capacity-based tariff. An energy storage system (ESS) is installed to keep the power at the point of connection small. On site there may also be a renewable generator.

Variables and model. With the *consumer sign convention* — power consumed is positive — the quantities at time i of the window opened at k are: the charging demand of the vehicles $p^{pev}_{i|k}$ and the renewable generation $p^{res}_{i|k}$, both known over the window (disturbances with a forecast); the storage power $u_{i|k}$, positive when charging, which is the control; the energy stored $x_{i|k}$, which is the state; and the power at the point of connection $p_{i|k}$, which is the controlled quantity. The balance at the point of connection and the dynamics of the storage are

$$p_{i|k} = p^{pev}_{i|k} + u_{i|k} - p^{res}_{i|k}, \qquad x_{i+1|k} = x_{i|k} + \tfrac{T}{3600}\, u_{i|k}, \qquad x_{0|k} = x_k. \tag{23}$$

Problem solved at every step.

$$\begin{aligned} \min_{u} \quad & \sum_{i=0}^{N-1} \left[\alpha\, p^2_{i|k} + \beta \left(x_{i|k} - x^{ref} \right)^2 \right] + \beta_f \left(x_{N|k} - x^{ref} \right)^2 \\ \text{s.t.} \quad & \text{balance, dynamics, } x_{0|k} = x_k, \\ & u^{\min} \le u_{i|k} \le u^{\max}, \quad x^{\min} \le x_{i|k} \le x^{\max}, \quad p^{\min} \le p_{i|k} \le p^{\max}. \end{aligned} \tag{24}$$

The weight α penalizes the power at the point of connection; β, deliberately small, keeps the storage near a reference level so that it is ready for the next event; the terminal weight β_f is taken from the Riccati equation of the unconstrained problem, as section 3 prescribes. Since p is an affine function of u, the limits on p are bounds on the control, and the problem is a quadratic program.

Losses and indicators. A real storage has a charging and a discharging efficiency, and the two cases obey different equations. The storage power is split into a charging and a discharging power, $p^{ess} = p^{ch} - p^{dis}$, both non-negative, with the indicator variables $\delta^{ch}, \delta^{dis}$, the constraints $p^{ch} \le p^{ch,\max}\delta^{ch}$, $p^{dis} \le p^{dis,\max}\delta^{dis}$, $\delta^{ch} + \delta^{dis} \le 1$, and the dynamics with the efficiencies, as in section 7. The problem becomes a mixed-integer quadratic program. We also distinguished the *control model*, which lives inside the MPC problem and is the controller's belief about the plant, from the *simulation model*, which replaces the real system when the controller is tested.

Code. The implementation is split in two files on purpose: `main.jl` is the application, which holds the parameters, builds the profiles and runs the receding-horizon loop — measure, call the optimizer, apply the first control, simulate one step, repeat; `MPC_iteration.jl` is the controller, which builds the optimization problem in the modelling language JuMP and hands it to a solver. Changing solver is one line, and the model does not change: this separation between how a problem is written and how it is solved is the reason for using a modelling language at all.

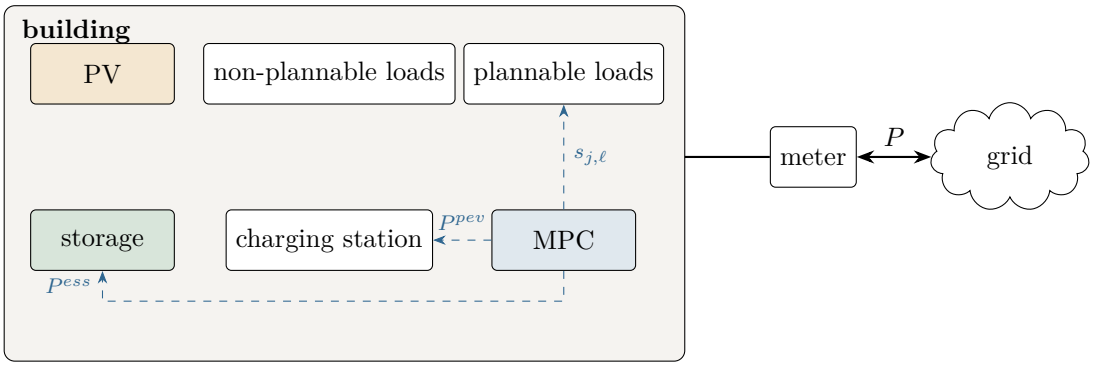


Figure 8: The building of the second case study. The controller acts on the storage, on the charging station and on the start times of the plannable loads; generation and non-plannable loads are forecasts; the bill is paid on the power at the meter.

On the course website

The Service Area Lab runs the controller over a full day against the same area without storage; the peak at the point of connection halves. The lab also shows the effect of the horizon, of the forecast, and of the terminal weight. The Julia code of the lesson is on the website, with the formulation it implements. Among real installations, battery-buffered fast chargers sold for sites with a weak grid connection are this problem turned into a product.

8.2 Building energy management

Problem. A building has photovoltaic panels, non-plannable loads, plannable loads, an energy storage system and a charging station, all behind one meter at the point of delivery (fig. 8). The first goal is to minimize the electricity bill under a time-of-use tariff, with one price c^{in} for the energy consumed and a lower price c^{out} for the energy injected into the grid.

Plannable loads. Each plannable load ℓ comes with the preferences of the user, the earliest start S_ℓ, the deadline F_ℓ, the duration N_ℓ, and with its power profile P_ℓ. Load shifting is the decision of when to start it, written with the one-hot variables $s_{j|k,\ell}$ of section 7: the load starts exactly once, $\sum_{j=S_\ell}^{F_\ell-N_\ell} s_{j|k,\ell} = 1$, its power is the shifted profile $P^{pl}_{i|k,\ell} = \sum_j s_{j|k,\ell}\, P_\ell(i-j+1)$, and the total plannable power is $P^{pl}_{i|k} = \sum_\ell P^{pl}_{i|k,\ell}$. Note the two time indices: S_ℓ and F_ℓ are absolute times, $i|k$ is relative to the window.

The bill. The power at the meter, $P_{i|k} = P^{npl} + P^{pl} + P^{pev} + P^{ess} - P^{pv}$, is a cost when positive and a revenue when negative. It is split as $P = P^{in} - P^{out}$ with $P^{in}, P^{out} \geq 0$, the two forbidden from being positive together by the indicators $\delta^{in}, \delta^{out}$ and the constraints $P^{in} \leq P^{\max}\delta^{in}$, $P^{out} \leq |P^{\min}|\,\delta^{out}$, $\delta^{in} + \delta^{out} \leq 1$. The bill over the window is then linear:

$$J^{cost} = \sum_{i=0}^{N-1} \frac{T}{3600}\left(c^{in}_{i|k} P^{in}_{i|k} - c^{out}_{i|k} P^{out}_{i|k}\right). \tag{25}$$

Battery ageing. A storage system has a price $COST^{ess}$ and a finite number of full cycles. The energy it will move over its life is $E^{life} = 2\, x^{ess,\max} \cdot cycles$, and the ratio $c^{dep} = COST^{ess}/E^{life}$ is what one kWh through the battery costs in terms of life consumed — a few

cents. Both charging and discharging consume life, so the term is

$$J^{depr} = \sum_{i=0}^{N-1} c^{dep} \left(P^{ch}_{i|k} + P^{dis}_{i|k}\right) \tfrac{T}{3600}, \tag{26}$$

which is $c^{dep}\,|P^{ess}|$ written without an absolute value, thanks to the split into two non-negative powers. It acts as a friction: the storage is used only when the benefit exceeds the loss of life.

The full objective. With a regulation term on the stored energy and the terminal value of section 6,

$$J = J^{cost} + \alpha_x \sum_i \left(x^{ess}_{i|k} - x^{ref}\right)^2 + \alpha_{dep}\, J^{depr} - c^{avg}\, x^{ess}_{N|k}, \tag{27}$$

subject to the balance at the meter, the dynamics of the storage with its efficiencies, the measured initial state, the one-hot constraints of the loads, the indicator constraints and the bounds on all variables. Without the last term the battery ends every day empty, because selling its energy is a profit inside the window; with it, the energy left is valued at the average purchase price and the controller keeps it. The weights are chosen as in section 6, and they can be normalized to sum to one.

What was left out. The model is electrical only. A real battery management system also tracks the state of charge in percent, the state of health, which cell balancing protects, and the state of power, the power the battery can deliver in the near future. The thermal part of the building — wall and room temperatures driven by the weather and by heating and cooling, humidity, air quality, ventilation — would add comfort terms to the objective and constraints of its own. The building information model is the source from which the parameters of all these models can be updated as the building changes.

On the course website

The Building Energy Lab runs this problem on a detached house with five appliances under a time-of-use tariff, and compares the plan with an uncontrolled house and with a rule-based battery. Two experiments belong to this chapter: removing the terminal value and watching the battery end the day empty, and asking the house for a flexibility service — a power cap or a power curve — with a fee for delivering it and a penalty for failing. Measured demonstrations of predictive control on real buildings exist, on offices, with savings of the order of tens of percent on the energy for heating and cooling.

8.3 An aggregator of electric vehicles

Context. Electricity is traded on wholesale markets by generators, retailers and the market operator. Next to them, the transmission and distribution system operators buy on the *ancillary services* or *flexibility* markets what they need to keep frequency and voltage at their references. The providers are large consumers or *aggregators* of many small resources, managed as one entity to reach the minimum bidding size — of the order of a megawatt for a given window. A fleet of charging vehicles is such an aggregation: the goal of the aggregator is to make the *total* charging power of the fleet track a profile agreed with the market, while every driver still receives the charge requested.

Model. For each vehicle p connected during the window, a charging and a discharging power, $P^{ch}_{i|k,p}, P^{dis}_{i|k,p} \geq 0$, with their indicators and mutual exclusion; the stored energy with the efficiencies, $x_{i+1|k,p} = x_{i|k,p} + \eta^{ch}\, T\, P^{ch}_{i|k,p} - \frac{1}{\eta^{dis}}\, T\, P^{dis}_{i|k,p}$; and the fact that a charger cannot

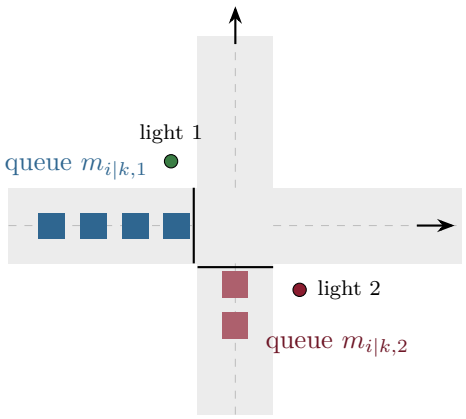


Figure 9: The intersection of the fourth case study: two one-way roads, one light each, queues counted at the stop lines.

deliver an arbitrarily small power, the semi-continuous constraint (18), $\delta^{ch}_{i|k,p} P_p^{ch,\min} \le P^{ch}_{i|k,p} \le \delta^{ch}_{i|k,p} P_p^{\max}$. The total power is $P_{i|k} = \sum_p \left(P^{ch}_{i|k,p} - P^{dis}_{i|k,p}\right)$.

User preferences. Each driver states a final charging time F_p and a desired state of charge x_p^{ref}. If the vehicle leaves inside the window, $F_p \le k+N$, the request is a constraint, $x_{F_p|k,p} \ge x_p^{ref}$; if it leaves after the window, the request enters the cost as a term $(x_{N|k,p} - x_p^{ref})^2$. The constraint can be impossible to meet — a vehicle that arrives too empty or stays too little — and it is therefore *relaxed* as in section 7.7:

$$x_{F_p|k,p} \ge x_p^{ref} - \varepsilon_p, \qquad \varepsilon_p \ge 0. \tag{28}$$

Objective.

$$J = d_1 \sum_{i=0}^{N-1} \left(P_{i|k} - P^{ref}_{i|k}\right)^2 + d_2 \sum_{p:\, F_p > k+N} \left(x_{N|k,p} - x_p^{ref}\right)^2 + d_3 \sum_p \varepsilon_p^2. \tag{29}$$

The weight d_3 decides the behaviour: small, and the controller disappoints a driver to follow the grid better; large, and it disappoints a driver only when the request cannot be met at all. Sweeping it traces the Pareto frontier between the two goals.

On the course website

The PEV Aggregator Lab runs the problem in closed loop over a day, with the request for a stop in the aggregate power arriving at a chosen hour: until then the controller spreads the charge, and at that hour the plan is rearranged. In Italy the role described here is regulated: Terna's UVAM pilot admits aggregates of loads, storage and vehicles of at least one megawatt into the balancing market, paid for availability and for the energy actually moved.

8.4 Traffic lights at an intersection

Problem. One intersection with a set $\mathcal{J}$ of traffic lights, one per approaching road (fig. 9). Two goals belong to two different people: the traffic manager wants short queues, the drivers want short waiting times.

Control variables. The colours: for each light J three binaries $g_{i|k,J}, y_{i|k,J}, r_{i|k,J} \in \{0,1\}$, exactly one of which is on,

$$g_{i|k,J} + y_{i|k,J} + r_{i|k,J} = 1. \tag{30}$$

The admissible transitions are green → yellow → red → green, and each forbidden transition is a linear inequality: green cannot be followed by red, red by yellow, yellow by green,

$$r_{i+1|k,J} \le 1 - g_{i|k,J}, \qquad y_{i+1|k,J} \le 1 - r_{i|k,J}, \qquad g_{i+1|k,J} \le 1 - y_{i|k,J}. \tag{31}$$

The same inequalities, written between the first step of the window and the colour actually displayed at $k-1$, tie the plan to the current state of the lights. Conflicting lights cannot be green or yellow at the same time, $\sum_J \left(g_{i|k,J} + y_{i|k,J}\right) \le 1$.

The yellow. Safety requires a minimum duration of the yellow. A timer counts how long the yellow has been on, $t^y_{i+1|k,J} = \left(t^y_{i|k,J} + T\right) y_{i|k,J}$, a product of a real variable and a binary linearized as in section 7.5, and the light cannot turn red before the timer reaches the minimum:

$$r_{i+1|k,J} \le \frac{t^y_{i|k,J}}{t^{y,\min}_J} + \left(1 - y_{i|k,J}\right). \tag{32}$$

While the timer is short of the minimum the first term is strictly less than one, and since r is binary the constraint forces $r_{i+1|k,J} = 0$; the second term makes the constraint irrelevant when the light is not yellow.

Queues and waiting times. Two models. The *aggregate* model keeps one state per light, the number of vehicles waiting, with the dynamics

$$m_{i+1|k,J} = m_{i|k,J} + v^{in}_{i|k,J}\, T - v^{out}_J\, T\, g_{i|k,J}, \qquad m_{0|k,J} = m_{k,J} \text{ measured}, \quad m \ge 0, \tag{33}$$

where v^{in} is the arrival rate and v^{out} the discharge rate on green; the constraint $m \ge 0$ is an approximation of the fact that a queue cannot become negative. Average waiting times then follow from Little's law. The *detailed* model keeps position, speed and acceleration of every vehicle, the acceleration being a model of the driver for human-driven cars and a command for automated ones, and a binary $\delta_{i|k,v}$ that says whether the vehicle is still in the queue, so that its waiting time grows as $w_{i+1|k,v} = \left(w_{i|k,v} + T\right) \delta_{i|k,v}$ and the queue length is the sum of the indicators.

Objective. On the queues, $J^m = \sum_i \sum_J m^2_{i|k,J}$: the square shortens the queues and balances them, since for a given total it prefers two queues of five to one of ten. On the waiting times the goal is to minimize the *largest* of them, written with an epigraph variable $\bar{w}_{i|k}$, $w_{i|k,v} \le \bar{w}_{i|k}$ for every vehicle present, and $J^w = \sum_i \bar{w}^2_{i|k}$ in the cost. The problem is a mixed-integer quadratic program; a city is many intersections tied by coupling constraints, which is where centralized and distributed MPC return.

On the course website

The Traffic Light Lab runs the intersection of fig. 9 with the same vehicles arriving on both sides of the screen, a fixed-time plan on the left and the MPC on the right, with queues and waiting times compared as the day goes on. With two lights the transition constraints leave few enough admissible colour sequences over the window for the page to enumerate them and find the best one exactly; the mixed-integer formulation is what is needed when the intersection is larger. Systems of this kind run in cities: Surtrac in Pittsburgh re-plans every intersection on a rolling horizon about once a second, and UTOPIA, born in Turin,

has been in operation since the nineteen-eighties.

8.5 Task scheduling in an assembly line

This section covers what was done in class up to the lesson of 26 October; it will be completed after the next lesson.

Problem. A *job* — the assembly of a vehicle — is made of elementary *tasks*, executed on *workstations* and requiring *resources*: tools, people, energy. Among the possible goals — cost, quality, scrap, balanced use of the workstations — the one taken here is the total completion time of the job, the *makespan*, read on a Gantt chart. The orders of magnitude are what makes the problem hard: ten thousand tasks over sixty days.

Sets and parameters. The tasks $\mathcal{T}$, the workstations $\mathcal{W}$ and, for each task, the subset $\mathcal{W}_t \subseteq \mathcal{W}$ of workstations that can execute it. Each task t carries an earliest start S_t, a deadline F_t, an estimate d_t of the number of steps it needs, and the number $R^{\#}_{t,r}$ of resources of each type $r \in \mathcal{R}$ it consumes; R^{a}_{r} is the number available.

Decision variables and dynamics. One binary per task, workstation and time, $e_{i|k,t,w} \in \{0,1\}$, equal to one when task t runs at time $i|k$ on workstation w. A task runs on at most one workstation at a time,

$$\sum_{w \in \mathcal{W}_t} e_{i|k,t,w} \leq 1, \qquad e_{i|k,t} = \sum_{w \in \mathcal{W}_t} e_{i|k,t,w} \in \{0,1\}, \tag{34}$$

where the auxiliary $e_{i|k,t}$ says whether the task is running. The state is the time left to complete each task, decremented by one unit for every step actually worked,

$$d_{i+1|k,t} = d_{i|k,t} - \sum_{w \in \mathcal{W}_t} e_{i|k,t,w}, \tag{35}$$

and the deadline is $d_{F_t,t} = 0$ for every task whose deadline falls inside the window.

Precedence constraints. The relations that make a schedule a schedule: finish-to-start, finish-to-finish, start-to-start, start-to-finish. The first, task t_2 cannot run until t_1 is finished, is the one to be written as an inequality in the next lesson, together with the resource constraints and the objective.

Further reading

For the modelling constructions of all five case studies, [Hax_1977], chapter 9, and [Borrelli_2017]. For the market context of the aggregator, [Kirschen_2019], chapter 3 on markets for electrical energy and chapter 6 on power system operation; background reading. For the traffic and the scheduling applications, the papers [Liberati_2025a] and [Liberati_2025b] show the same formulations on real problems; background reading, not examination material.

References

The keys are the same as on the course website, where the links are kept up to date.

[Cannon_2016]
B. Kouvaritakis, M. Cannon. *Model Predictive Control: Classical, Robust and Stochastic*. Springer, 2016. Main reference of the course: chapter 2 follows the same route, and the same notation, as these notes.

[Rawlings_2017]
J. B. Rawlings, D. Q. Mayne, M. Diehl. *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017. Free PDF from the authors.

[Borrelli_2017]
F. Borrelli, A. Bemporad, M. Morari. *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.

[Boyd_2004]
S. Boyd, L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. Free PDF from the authors.

[Hax_1977]
S. P. Bradley, A. C. Hax, T. L. Magnanti. *Applied Mathematical Programming*. Addison-Wesley, 1977. Chapter 9.

[Kirschen_2019]
D. S. Kirschen, G. Strbac. *Fundamentals of Power System Economics*, 2nd ed. Wiley, 2019.

[Mayne_2000]
D. Q. Mayne, J. B. Rawlings, C. V. Rao, P. O. M. Scokaert. "Constrained model predictive control: Stability and optimality." *Automatica* 36(6), 789–814, 2000.

[Liberati_2025a]
F. Liberati, M. Donsante, A. Tortorelli. "Single intersection MPC traffic signal control in presence of automated vehicles." *IEEE Transactions on Control Systems Technology* 33(4), 1432–1446, 2025.

[Liberati_2025b]
F. Liberati, M. Donsante, C. M. F. Cirino, A. Tortorelli. "Fast task scheduling with model predictive control integrating a priority-based heuristic." *IEEE Access* 13, 14499–14515, 2025.